

Comparing Optimization Algorithms for the N-Queens Problem

Marcelo Augusto de Azevedo Freitas Filho
Univ. of Europe for Applied Sciences
Konrad-Ruse Ring 11, 14469 Potsdam, Germany.
marcelo.deazevedo@ue-germany.de

Raja Hashim Ali
Department of Business
AI Research Group
Univ. of Europe for Applied Sciences
Konrad-Ruse Ring 11, 14469 Potsdam, Germany.
hashim.ali@ue-germany.de

Abstract—The N-Queens problem is a classic challenge in pattern recognition and artificial intelligence, consisting of placing N queens on an $N \times N$ chessboard so that none of them is in the line of attack of another, meaning none can share the same row, column, or diagonal. The complexity lies in the difficulty of constraint satisfaction and searching in large-dimensional solution spaces. This problem is widely used as a benchmark in artificial intelligence because it reflects well the core difficulty of combinatorial optimization problems. Despite the existence of an extensive literature, there is still a limitation in the systematic comparison between exhaustive and heuristic methods for different board sizes regarding the trade-offs between solution quality, computational time, and memory consumption. In this study we evaluate four algorithms — Depth-First Search (DFS), Hill Climbing, Simulated Annealing, and Genetic Algorithm — for board sizes between N equal to 10, 30, 50, 100, 200, and 500. The experimental results show that DFS guarantees exact solutions but becomes computationally infeasible for N greater than 30, while other methods scale to larger instances at some cost in solution quality. These results offer practical guidance and better understanding for the selection of algorithms in constraint-based optimization tasks, with direct implications in the trade-off between accuracy and scalability for algorithms and applications of artificial intelligence.

Index Terms—N-queens problem, optimization algorithms, genetic algorithm, exhaustive search technique, local search techniques

I. INTRODUCTION

Optimization is one of the greatest challenges in computing and artificial intelligence, since many real-world problems require finding the best possible solution in a candidate space that can be enormous. From route planning to task scheduling, intelligent systems need to search efficiently in complex solution spaces and do so in a viable time. Exhaustive approaches, despite guaranteeing the correct solution, basically try all existing possibilities, which quickly becomes impractical as the problem grows. For this reason, smarter strategies have emerged that give up always finding the perfect solution, but gain greatly in speed and performance. Among these strategies are methods such as local search, simulated annealing, and genetic algorithms, widely used in optimization problems. In pattern recognition, these methods can be used in tasks such as feature selection, parameter optimization, and tackling problems that are too hard to solve by hand. In the end,

choosing the right algorithm depends greatly on the size of the problem and what is available in terms of time and memory.

The N-Queens problem is an interesting case study for comparing optimization algorithms because, despite being relatively simple to understand, it becomes extremely difficult to solve at scale. The goal is to position N queens on an $N \times N$ board so that none of them can attack another, meaning none can share the same row, column, or diagonal. For small values of N, the Depth-First Search (DFS) manages to find the correct solution by trying different positions and going back whenever it encounters a conflict. The problem is that this grows very fast — for large N, DFS simply cannot finish in reasonable time. That is where heuristic methods come in, working in a very different way — Hill Climbing adjusts the queen that generates the most conflicts until it finds a better position, Simulated Annealing does the same but occasionally accepts a worse position to avoid getting stuck, and the Genetic Algorithm works with several solutions at the same time and adapts them based on the best previous results. Although these methods do not always guarantee the perfect solution, they manage to find very good results in much shorter times. Throughout this study, the four methods are compared for board sizes between $N=10$ and $N=500$, considering execution time, memory consumption, and quality of the solutions found.

A. Related Work

Genetic algorithms appear frequently in works on the N-Queens problem because they handle much larger search spaces than exhaustive methods. Garg et al. [1] tried to improve the diversity of solutions using a hybrid crossover operator, although the article also shows that the computational cost continues to grow considerably for high values of N. In another work, Cao et al. [2] focused more on performance and parallelized the genetic algorithm with MPI and GPU, managing to considerably reduce execution time for larger instances. Sharma and Jain [3] followed a similar line, but focused on the mutation operator to avoid premature convergence of solutions. Bell and Stevens [4] carried out a fairly complete survey on the N-Queens problem and remain an important reference to this day. More recently, Moghimi and Amini [5] presented an approach based on non-sequential conflict resolution that showed competitive results for larger values of N compared

to more traditional methods. Alizadehbirjandi et al. [6], in a recent work from this university, directly compared exhaustive search with an improved genetic algorithm for different board sizes, serving as a direct reference for the present study. Table I summarises the most relevant works considered in this study.

TABLE I
SUMMARY OF RELATED WORKS IN THE LITERATURE

Author(s)	Year	Method	Key Findings
Garg et al.	2022	GA with hybrid crossover	Effective but costly for large N
Cao et al.	2020	Parallel GA (MPI-CUDA)	Better accuracy via parallelization
Sharma & Jain	2021	GA with novel mutation	Increased solution diversity
Bell & Stevens	2009	N-Queens survey	Comprehensive overview
Moghimi & Amini	2024	Non-sequential conflict resolution	Efficient for large N
Alizadehbirjandi et al.	2025	Exhaustive search + improved GA	Direct DFS vs GA comparison

B. Gap Analysis

Despite the significant number of works on the N-Queens problem, there are still few direct comparisons between exhaustive and heuristic methods considering different board sizes and using the same evaluation metrics. What can be observed is that most works focus on a single method and go deep into it, without comparing with others. Many studies also only test for small N — when N reaches 200 or 500, there is not much published. Without this, it is hard to know in practice which algorithm would hold up better when the problem scales. Another problem is that it is rare to find a study that measures time, memory, and solution quality at the same time. That is exactly what we tried to do here — compare the four methods under the same conditions and see what happens when N gets large, using standardised metrics and evaluating the behaviour of the algorithms at different scales of the problem.

C. Problem Statement

The N-Queens problem consists of positioning N queens on an $N \times N$ chessboard so that none of them attacks another. Two queens attack each other when they occupy the same row, column, or diagonal of the board. In practice, what is wanted is to fit all the queens on the board without any of them being able to reach another — and as N changes, the number of possible solutions also changes considerably. What surprises many people is that even though it seems simple, when N starts to grow the number of possible combinations becomes absurdly large very quickly. In the end, it is a problem that ends up appearing a lot in AI work precisely because it makes it clear where each algorithm starts to have difficulties.

In the case of the 10-Queens, the challenge is to fit 10 queens on a 10×10 board without any of them attacking another. Figure 1 shows three situations: first the empty board, then an attempt with errors where queens in conflict can be seen highlighted in red, and finally a configuration that works

with all positions correct. For small N it is still possible to explore everything with exhaustive methods, but when N grows this stops working and smarter methods are needed to reach an answer in useful time.

Following are the main questions addressed in this report.

- 1) How to develop a solution using Depth-First Search (DFS)?
- 2) How to implement approaches based on Hill Climbing and Simulated Annealing?
- 3) How to develop a solution using Genetic Algorithms?
- 4) How to compare the four methods in terms of execution time, memory consumption, and solution quality for N equal to 10, 30, 50, 100, 200, and 500?

D. Novelty of our work

The main contribution of this study lies in the implementation and direct comparison of four optimization algorithms — DFS, Hill Climbing, Simulated Annealing, and Genetic Algorithm — applied to the N-Queens problem for values of N between 10 and 500. The evaluation was carried out using three main metrics: execution time, memory consumption, and number of conflicts present in the solution found. Unlike many existing works, which usually analyse only a specific algorithm or variations within the same approach, this study compares methods of quite different natures, including exhaustive search, local heuristics, and evolutionary algorithms. This helps to understand from what point each method starts to have difficulties. Another care taken was to adjust the parameters of each algorithm according to the size of N — without this the comparison would not be fair. In the end, what we want to understand is which algorithm makes more sense to use depending on the size of the problem and the time available.

E. Our Solutions

In this study, four algorithms were implemented in Python to solve the N-Queens problem. The first method was Depth-First Search (DFS) with backtracking, which explores the solution space by placing queens column by column and goes back whenever it encounters a conflict. A Hill Climbing algorithm with random restarts was also developed, starting from random configurations and trying to gradually reduce the conflicts on the board. Simulated Annealing follows a similar idea, but occasionally allows moves to worse states in order to avoid local optima during the search. Finally, a Genetic Algorithm was implemented using tournament selection, one-point crossover, random mutation, and elitism. All four methods were evaluated for N values of 10, 30, 50, 100, 200, and 500, considering execution time, memory consumption, and number of conflicts in the solutions found.

When comparing the results, what becomes most evident are the differences in scalability between the methods. DFS can find exact solutions quickly for smaller values of N, but execution time grows very steeply as the problem increases, making the approach less viable for larger instances. Hill Climbing showed good results for small and medium boards, although it started having more difficulty completely reducing

N-Queens Problem — 10×10 Board

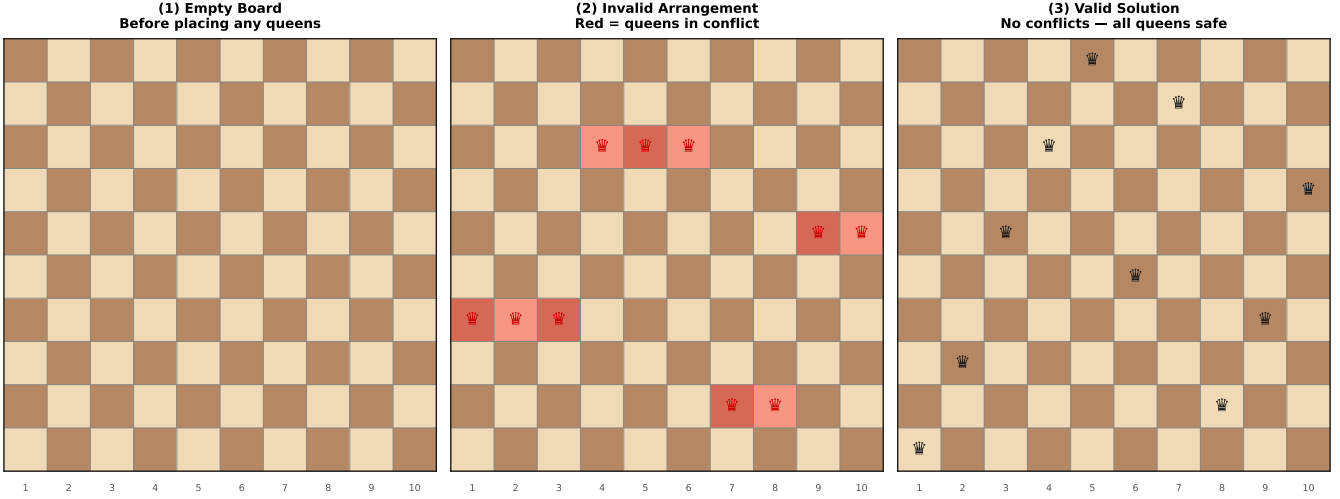


Fig. 1. (1) Empty 10x10 chessboard before placing any queens, (2) invalid arrangement with queens highlighted in red showing conflicts in the same row, and (3) valid solution with all 10 queens correctly placed without any conflicts.

conflicts for high values of N . Simulated Annealing showed more stable behaviour for intermediate instances, managing to find solutions close to optimal in reasonable times. The Genetic Algorithm was the only method capable of producing solutions for all tested sizes, including $N=500$, despite the higher computational cost per iteration. In general, it became clear that for large problems heuristic and evolutionary methods make much more sense — DFS simply cannot keep up.

II. METHODOLOGY

A. Overall Workflow

The methodology of this study was organised in four main phases, as shown in Figure 2. In the first phase, the N-Queens problem was represented in the same way for all algorithms through a list of N integers, where the index represents the column of the board and the value corresponds to the row in which the queen is positioned. This representation simplifies conflict detection and also reduces the search space, since it automatically guarantees one queen per column. In the second phase, each of the four algorithms was initialised with its respective parameters and executed for the six board sizes considered in the study. In the third phase three metrics were collected during the executions: execution time in seconds measured with the `perf_counter` function from the `time` library, memory consumption in megabytes obtained through the `trace-malloc` library, and the number of residual conflicts present in the solution obtained. Finally, in the fourth phase the results were organised into tables and presented in comparative graphs to facilitate the analysis of differences between the methods. In the case of DFS a maximum limit of 30 seconds per instance was defined and when this limit was exceeded the execution was interrupted and the result recorded as timeout.

TABLE II
HYPERPARAMETER CONFIGURATION PER ALGORITHM

Algorithm	Parameter	$N \leq 50$	$N \leq 200$	$N = 500$
DFS	Time limit	30s	T/O	T/O
Hill Climbing	Max restarts	200	20	—
Hill Climbing	Max steps	N^2	N^2	—
Sim. Annealing	Initial T	5.0	10.0	20.0
Sim. Annealing	Min T	0.001	0.0001	0.0001
Sim. Annealing	Cooling rate	0.9995	0.9998	0.9999
Genetic Alg.	Population	50	100	200
Genetic Alg.	Max generations	500	1000	2000
Genetic Alg.	Mutation rate	0.05	0.05	0.10

B. Experimental Settings

Table II presents the hyperparameters used in each algorithm for the different board sizes evaluated in the study. These parameters were adjusted taking into account the behaviour of each method, seeking to maintain a balance between solution quality and execution time. The objective was to ensure that all algorithms could be run under comparable conditions, especially for higher values of N , where the computational cost starts to weigh more.

III. RESULTS

A. Depth-First Search (DFS)

The DFS algorithm with backtracking was executed for all board sizes considered in the study, using a maximum limit of 30 seconds per instance. For $N=10$, the algorithm found a valid solution with no conflicts in approximately 0.0015 seconds, using around 0.0012 MB of memory. For $N=30$, it was also possible to find a correct solution, although execution time increased significantly to around 20 seconds. From $N=50$ onwards, the algorithm was no longer able to find solutions within the established time limit, resulting in timeout for $N=50$, 100, 200, and 500. In practice, DFS simply does not

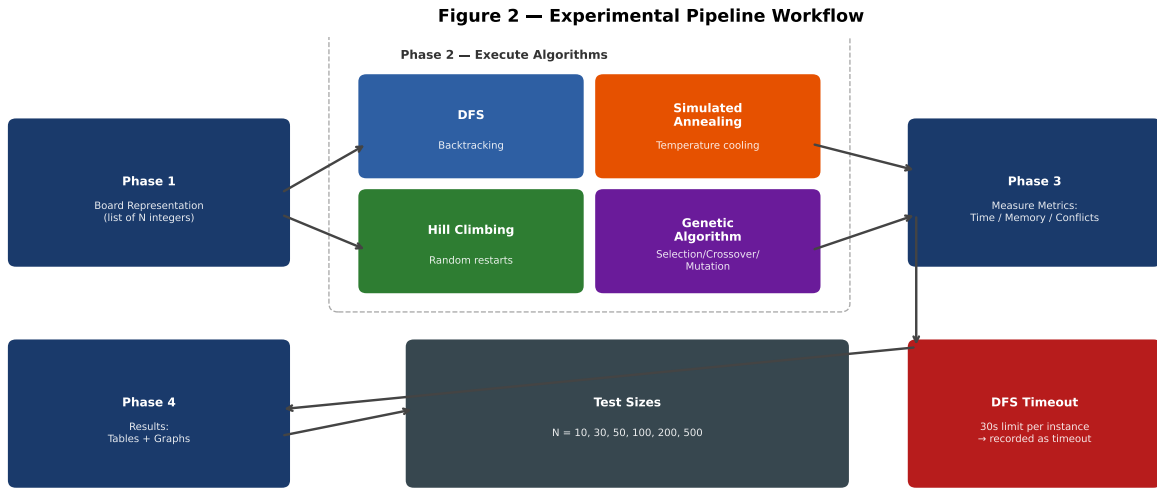


Fig. 2. Workflow for solving the N-Queens problem using the four algorithms evaluated in this study, organised in four phases: board representation, algorithm execution, metric collection, and results analysis.

work for large N — the numbers make this very clear. In Figure 3 it is easy to see how time shoots up as N grows.

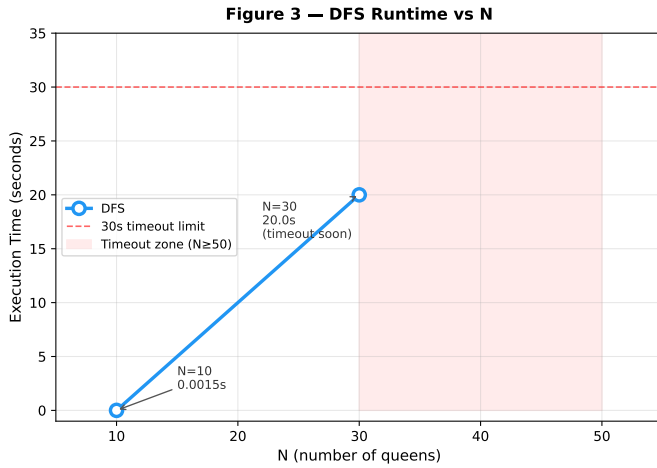


Fig. 3. Runtime of the DFS algorithm as a function of the number of queens (N). The dashed red line marks the 30-second timeout limit, beyond which the algorithm is interrupted.

B. Hill Climbing and Simulated Annealing

Hill Climbing with random restarts showed good results for smaller values of N , finding solutions with no conflicts for $N=10$ in 0.003 seconds and for $N=30$ in approximately 0.15 seconds. For $N=50$ and $N=100$, the algorithm continued to produce solutions relatively close to optimal, although it already showed some residual conflicts and a considerable increase in execution time. In tests for $N=200$ and $N=500$, the

method started showing more evident limitations, exceeding the practical limits defined for the study. Simulated Annealing held up better for intermediate instances. The algorithm managed to find solutions with no conflicts for $N=10$ and $N=50$, as well as solutions with few conflicts for $N=30$ and $N=100$. For $N=200$, it was still possible to obtain a solution in reasonable time, although with a larger number of residual conflicts. For $N=500$, the method also exceeded the time limit in the configurations used. Table III and Figure 4 show the complete results of both algorithms.

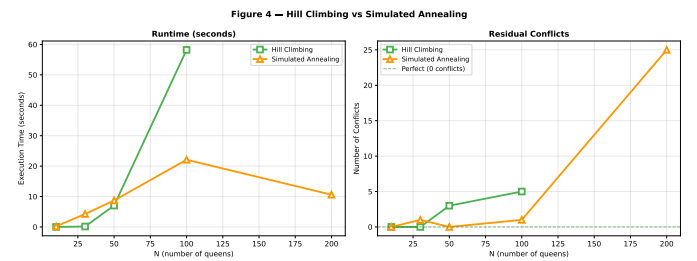


Fig. 4. Runtime and residual conflicts comparison between Hill Climbing and Simulated Annealing. Simulated Annealing shows more consistent results for intermediate instances.

C. Genetic Algorithm

The Genetic Algorithm was the only method capable of producing solutions for all board sizes evaluated, including $N=500$. For smaller values of N , the algorithm found solutions relatively close to optimal in a few seconds, although not always with zero conflicts. The larger the board, the more conflicts were left behind — the GA progressively loses

quality as N grows. Even so, the algorithm maintained acceptable execution times for all tests, showing that it scales much better than the others. The memory consumption of the Genetic Algorithm was consistently higher, mainly due to the storage of the population of candidate solutions throughout the generations. In Figure 5 it is possible to follow how the execution time and residual conflicts of the GA vary for the different values of N .

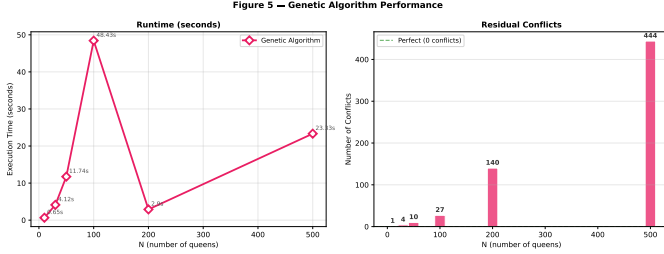


Fig. 5. Runtime and residual conflicts of the Genetic Algorithm for board sizes from $N=10$ to $N=500$. The algorithm is the only method capable of producing solutions for all tested sizes, though solution quality decreases for larger N .

IV. ANALYSIS

The results obtained followed relatively well the behaviour expected by the theoretical complexity of each algorithm. In the case of DFS, the factorial complexity $O(N!)$ causes the number of combinations to grow very rapidly as new columns are added to the board. This became quite evident during the tests. For $N=10$ the algorithm finished practically immediately, while for $N=30$ the time was already close to 20 seconds. After that the computational cost grows too fast for the method to remain usable within the limits defined in this work. Hill Climbing and Simulated Annealing behaved differently because both work with local search and depend much more on the number of steps executed during optimisation than on the complete space of combinations. Even for larger values of N , both still managed to produce results in times considered reasonable. In the case of the Genetic Algorithm, the computational cost was directly linked to the population size and the number of generations used. This ended up providing more flexibility to control the balance between solution quality and execution time.

The efficiency differences between the methods also became quite visible throughout the tests. For small instances, DFS used little memory and found exact solutions quickly. The problem started appearing when the board size increased, since the time required grew very rapidly. Hill Climbing was the fastest method for small and medium instances, solving $N=30$ in a few tenths of a second, but started losing quality for larger values of N . In intermediate tests, Simulated Annealing maintained more stable behaviour and still managed to produce results for $N=200$, even with some residual conflicts. The Genetic Algorithm was the only method that continued working for all tested sizes, including $N=500$. On the other hand, it was also the method with the highest memory consumption,

mainly due to the simultaneous storage of multiple solutions during population evolution.

Regarding the precision of the solutions found, DFS remains the only method that truly guarantees a conflict-free answer whenever it manages to complete execution. In heuristic methods, solution quality started to decrease gradually as N increased. Hill Climbing started getting stuck more frequently in local minima, making it harder to completely eliminate conflicts. Simulated Annealing managed to handle this problem better in several intermediate instances because it allows less favourable moves during the search. The Genetic Algorithm was the most scalable method, but also the one that accumulated the most conflicts for higher values of N , reaching 444 conflicts for $N=500$. This shows that the choice of algorithm depends greatly on the objective of the application. When accuracy is the priority, DFS remains the most reliable approach. For larger problems where approximate solutions can be considered acceptable, heuristic and evolutionary methods turn out to be much more viable in practice.

V. DISCUSSION

The results obtained with Depth-First Search confirm what was already expected of DFS. The algorithm can guarantee a valid solution, but the time required grows very quickly as the board size increases. The difference observed between $N=10$, solved in less than 2 milliseconds, and $N=30$, which already required around 20 seconds, clearly shows this increase in complexity. From that point DFS simply does not finish because the combinations become too large. It makes sense that this happens since DFS tries everything and with each new column added the number of possibilities increases significantly, resulting in factorial complexity in the worst case. In practice this means the method is more suitable for small instances, mainly when there is a need to guarantee an exact solution.

The comparison between Hill Climbing and Simulated Annealing shows interesting differences between the two local search methods. Hill Climbing showed shorter execution times for small instances but started finding more difficulties as N increased, especially due to the tendency to get stuck in local minima. This became more evident in tests for $N=50$ and $N=100$ where the number of residual conflicts started to increase. Simulated Annealing managed to handle these cases better because it occasionally allows moves to worse states during the search, reducing the probability of premature convergence. That is why SA ended up finding solutions closer to optimal in several intermediate tests, although at a higher cost in execution time for small instances. For $N=200$ for example, SA still managed to produce a solution in reasonable time while Hill Climbing was already showing more evident limitations.

Of all the methods tested, GA was the one that held up best when N got large. It managed to reach $N=500$, being the only method capable of producing a solution for that size. However this capability came with a gradual increase in the number of residual conflicts as N grew. Part of this behaviour is related to

TABLE III
COMPLETE RESULTS — EXECUTION TIME (S) AND RESIDUAL CONFLICTS PER ALGORITHM AND BOARD SIZE

N	DFS		Hill Climbing		Simulated Annealing		Genetic Algorithm	
	Time (s)	Conflicts	Time (s)	Conflicts	Time (s)	Conflicts	Time (s)	Conflicts
10	0.002	0	0.003	0	0.20	0	0.65	1
30	20.0	0	0.15	0	4.27	1	4.12	4
50	T/O	—	7.04	3	8.70	0	11.74	10
100	T/O	—	58.22	5	22.1	1	48.43	27
200	T/O	—	T/O	—	10.6	25	2.90	140
500	T/O	—	T/O	—	T/O	—	23.33	444

T/O = timeout (exceeded 30s limit); — = not applicable

the parameters used during the tests, since larger populations and more generations would probably allow finding better solutions, although they would also increase the computational cost. Even so, the results show that GA can continue to operate in instances where the other methods have already stopped being viable.

Looking at the four methods together the differences become very clear. DFS guarantees the most exact solution but stalls for large N. Hill Climbing is fast and simple for small instances but gets stuck easily. Simulated Annealing balanced quality and performance better for intermediate instances. GA scales well but requires much more work to configure than the others. In the end each method has its place and the question is knowing when to use each one.

A. Future Directions

The results obtained in this study also open up several possibilities for future research. It may be worth exploring hybrid approaches that combine local search with evolutionary algorithms. Instead of starting the Genetic Algorithm with completely random solutions, solutions previously generated by Simulated Annealing could be used as a starting point, which would possibly improve the quality of results for larger instances. Parallelizing the algorithms taking advantage of multiple processing cores or GPU is another direction that would make sense to explore, especially for the Genetic Algorithm where larger populations imply higher computational cost. Automatic parameter tuning through techniques such as Bayesian optimization could also help to adapt each algorithm more efficiently for different board sizes without manual adjustment. Extending the comparisons from this study to other classic constraint satisfaction problems would help to verify whether the patterns observed here are repeated in problems with similar characteristics.

VI. CONCLUSION

This study compared four algorithms applied to the N-Queens problem: Depth-First Search (DFS), Hill Climbing, Simulated Annealing, and Genetic Algorithm. Tests were carried out for values of N between 10 and 500, considering execution time, memory used, and number of conflicts in the final solution. The results show quite large differences between the methods when the board size starts to increase. For small values of N, DFS can still find exact solutions very

quickly. However the computational cost grows so steeply that the approach stops being practical shortly after N=30 under the conditions used in this work. Hill Climbing performed quickly for small and medium instances, although in several cases it started getting stuck in local minima, mainly on larger boards. Simulated Annealing managed to handle this problem better because it occasionally allows less favourable moves during the search, reducing the probability of premature convergence. This ended up producing more stable solutions in some of the intermediate tests. The Genetic Algorithm was the only method that continued producing solutions for all sizes evaluated, including N=500. On the other hand the number of conflicts increased gradually as the problem grew, showing the difficulty of maintaining solution quality in very large instances. Overall the tests show that there is no single ideal method for all scenarios. DFS remains suitable when seeking accuracy in small instances, while Simulated Annealing showed a better balance for intermediate cases. For high values of N the Genetic Algorithm turned out to be the most viable approach within the practical limits defined in the study.

REFERENCES

- [1] P. Garg, S. S. Chauhan Gonder, and D. Singh, "Hybrid crossover operator in genetic algorithm for solving n-queens problem," in *Soft Computing: Theories and Applications*. Springer, 2022, pp. 91–99.
- [2] C. Jianli, C. Zhikui, W. Yuxin, and G. He, "Parallel genetic algorithm for n-queens problem based on message passing interface-compute unified device architecture," *Computational Intelligence*, vol. 36, no. 4, pp. 1621–1637, 2020.
- [3] S. Sharma and V. Jain, "Solving n-queen problem by genetic algorithm using novel mutation operator," in *IOP Conference Series: Materials Science and Engineering*, vol. 1116, no. 1, 2021, p. 012195.
- [4] J. Bell and B. Stevens, "A survey of known results and research areas for n-queens," *Discrete Mathematics*, vol. 309, no. 1, pp. 1–31, 2009.
- [5] O. Moghimi and A. Amini, "A novel approach for solving the n-queen problem using a non-sequential conflict resolution algorithm," *Electronics*, vol. 13, no. 20, p. 4065, 2024.
- [6] G. Alizadehbirjandi, R. H. Ali, R. Koutaly, T. A. Khan, and I. Ahmad, "Solving n-queens problem using exhaustive search and a novel genetic algorithm," in *Conference Paper*, 2025.